

Tabular Foundation Models: a TabPFN Case Study

Christopher Bonnelly, Bob de Boissvilliers

June 14, 2026

Abstract

So far the Transformer architecture has been applied heavily on text and images, and now on tabular data with TabPFN. The state-of-the-art methods for making predictions on tabular data were using AutoML and in particular AutoGluon [9]. The reason it has been so hard to apply deep learning methods to tabular data is fundamental: deep learning learns features from data, but in tabular data we already have the features, it is a very different problem to solve. Here are a few issues one could think of with tabular prediction before TabPFN: 1) often, free text and concepts are not leveraged 2) missing values are not natively treated 3) no semantic understanding of features and predictions 4) poor transfer of knowledge between datasets 5) poor outlier handling 6) often requires all data to be in one table 7) domain knowledge is inaccessible. TabPFN reframes tabular prediction entirely: rather than fitting a model per dataset, it approximates Bayesian inference in a single forward pass, having learned to do so across millions of synthetic datasets. This paper builds up the conceptual foundations needed to understand this idea, Bayesian inference, structural causal models, prior-data fitted networks, and then examines how TabPFN puts these pieces together.

1 Bayesian Inference

Before anything else, we need to understand what it means to reason under uncertainty in a principled way. Bayesian inference is the mathematical framework for doing exactly that, and it sits at the heart of everything TabPFN does.

1.1 The Core Idea

Suppose we are trying to understand some unknown mechanism ϕ that belongs to a broader space of possible mechanisms Φ . This ϕ could be the specific process that generated a dataset, the optimal parameters of a model, or any hidden quantity we care about, while Φ represents the entire hypothesis space of configurations under consideration. Bayesian inference provides the mathematical framework for updating our beliefs about ϕ within Φ as we observe data.

We start with a **prior** $p(\phi)$, which encodes our belief about how likely any given mechanism $\phi \in \Phi$ is before seeing any data. This is not a guess pulled from thin air: it reflects genuine prior knowledge, domain expertise, or deliberate structural assumptions. A well-chosen prior is powerful. A poorly chosen one can be too.

Once we observe a dataset D , we update our beliefs using Bayes’ rule:

$$p(\phi \mid D) = \frac{p(D \mid \phi) p(\phi)}{p(D)}.$$

The result, $p(\phi \mid D)$, is called the **posterior**: the probability distribution over the hypothesis space Φ after taking the data into account. The term $p(D \mid \phi)$ is the **likelihood**, which measures how probable the observed data would be if ϕ were the true underlying mechanism. The denominator $p(D)$ is a normalising constant that ensures the posterior integrates to one across the entire space Φ .

Intuitively: the posterior rewards individual hypotheses ϕ that were plausible *a priori* *and* that explain the data well. Occam’s razor is naturally built in: overly complex explanations that barely improve the likelihood get downweighted by the prior.

1.2 What ϕ and Φ Actually Are: Three Examples

The notation $p(\phi)$ over the space Φ is deliberately abstract. While that abstraction is its strength, it can also make the framework feel like symbols floating above reality. To ground this, it helps to see concretely what an individual mechanism ϕ and its overarching hypothesis space Φ look like in three different settings, moving from the familiar to the TabPFN-relevant.

Example 1: Linear regression. Here, an individual mechanism is a specific parameter configuration $\phi = (w, \sigma^2)$, where $w \in \mathbb{R}^d$ is a weight vector and $\sigma^2 > 0$ is a noise variance. The space of all possible hypotheses is $\Phi = \mathbb{R}^d \times \mathbb{R}_{>0}$, a perfectly ordinary Euclidean space. The prior $p(\phi)$ is a continuous probability density over this space; for instance, a Gaussian over w centred at zero expresses a belief that weights should be small (L_2 regularisation reread as a prior). The likelihood $p(D \mid \phi)$ evaluates the probability density of the observed targets given that specific choice of w and σ^2 . Bayes’ rule then yields a posterior density over weight vectors, updating which regions of our continuous space Φ are most plausible.

Example 2: Decision trees. Now, an individual ϕ is a single decision tree: a rigid binary tree structure paired with a feature index and a threshold at each internal node, alongside a class label at each leaf. The space Φ is no longer a smooth vector space—it is a massive, discrete, combinatorial set containing every valid tree topology and split combination imaginable. There is no natural notion of distance between two trees, and no gradient to follow. A prior $p(\phi)$ over this space assigns a discrete probability mass to each tree, perhaps assigning higher probability to shallower trees to penalise complexity. The likelihood $p(D \mid \phi)$ measures how accurately that specific tree structure classifies the training data. Here, computing the posterior requires evaluating probabilities over a combinatorial landscape, making exact inference intractable without approximations like MCMC.

Example 3: Structural Causal Models. In this setting, ϕ is an SCM: a directed acyclic graph over a set of variables, paired with a collection of functions f_i (mapping parents to children) and noise distributions $p(\epsilon_i)$. The hypothesis space Φ is a highly abstract space of *programs*. It contains objects with fundamentally different dimensions, topologies, and functional forms. A prior $p(\phi)$ over this space assigns probability to graph structures and functional families, explicitly preferring simpler graphs with fewer

edges. This is the exact prior used by TabPFN. Sampling $\phi \sim p(\phi)$ means drawing a random graph, drawing random functions for each edge, and drawing random noise distributions to yield one concrete data-generating mechanism. The likelihood $p(D \mid \phi)$ represents the probability that this specific causal program would output the exact data distribution seen in D .

The common thread across all three examples is that Φ always defines the sandbox of models we consider, and $p(\phi)$ always dictates which specific sandcastles we expect to see before data arrives. Keeping the distinct mathematical structures of Φ in mind makes the framework’s flexibility much less mystifying.

1.3 The Posterior Predictive Distribution

In supervised learning, our goal is not to identify the true mechanism ϕ , but to predict the label y_{test} associated with a new input x_{test} , given a labelled training dataset D_{train} . The Bayesian object that answers this question is the **Posterior Predictive Distribution** (PPD):

$$p(y_{\text{test}} \mid x_{\text{test}}, D_{\text{train}}) = \int_{\Phi} p(y_{\text{test}} \mid x_{\text{test}}, \phi) p(\phi \mid D_{\text{train}}) d\phi.$$

This expression should be read term by term.

The quantity

$$p(y_{\text{test}} \mid x_{\text{test}}, \phi)$$

is the probability of the test label *assuming a specific mechanism ϕ were the true data-generating process*. For a fixed ϕ , this is an ordinary predictive distribution: a linear model makes a Gaussian prediction, a classifier outputs class probabilities, a decision tree assigns a label distribution at a leaf.

The quantity

$$p(\phi \mid D_{\text{train}})$$

is the posterior over mechanisms: the probability that ϕ explains the training dataset well, given the prior assumptions encoded in $p(\phi)$. Mechanisms that fit the data poorly receive low weight; mechanisms that explain the data well receive high weight.

The integral

$$\int_{\Phi} (\cdot) d\phi$$

means that we do not select a single best mechanism. Instead, we *sum over all possible mechanisms*, weighting each one by how plausible it is after observing the training data. The integral notation is used for generality: when Φ is discrete, this is an ordinary sum; when Φ is continuous, it is an integral with respect to the appropriate base measure.

Putting these pieces together: the PPD is obtained by taking every possible hypothesis ϕ , asking what it would predict for x_{test} , weighting that prediction by how well ϕ explains the training data, and aggregating the result. The outcome is a full probability distribution over y_{test} .

This explains why the PPD represents uncertainty correctly. If many different mechanisms are consistent with the training data and they make different predictions, the PPD will reflect that disagreement. If the data strongly constrain the mechanism, the PPD will concentrate.

1.4 Why This Matters for Tabular Data

The PPD is especially appealing for tabular datasets because they are often small. With few training samples, any single model fit is unreliable: different reasonable models can give very different predictions. The PPD handles this gracefully by spreading probability over all consistent explanations rather than overcommitting to one. This is precisely the regime, small to medium datasets, where TabPFN shines.

2 Structural Causal Models and Bayesian Neural Networks

To approximate the PPD, we need to define the space of hypotheses Φ : what kind of mechanisms do we believe could have generated tabular data? TabPFN uses two complementary families of generative models as its prior: Structural Causal Models (SCMs) and Bayesian Neural Networks (BNNs).

2.1 Structural Causal Models

A Structural Causal Model (SCM) [1] specifies how each variable in a system is generated. It consists of a directed acyclic graph (DAG) G over variables $Z = \{z_1, \dots, z_k\}$, together with a set of structural assignments:

$$z_i = f_i(z_{\text{PA}_G(i)}, \epsilon_i).$$

This equation is read as follows. The variable z_i is generated by applying a deterministic function f_i to two inputs: the values of its parent variables $z_{\text{PA}_G(i)}$ in the graph, and an independent noise term ϵ_i . The parents represent direct causes of z_i , and the noise captures all unmodelled variation.

Sampling an SCM means generating data by executing these assignments. For each data point, one first samples all noise variables ϵ_i , then evaluates the equations in topological order of the graph, so that each variable z_i is computed from its parents and its noise. Repeating this process produces a dataset whose columns are causally related by construction. A synthetic tabular dataset is obtained by designating a subset of variables as features X and one variable as the target y , yielding rows that reflect the underlying causal structure encoded by the graph.

Why SCMs for tabular data? Because tabular data is often the result of a process that genuinely has causal structure. A patient’s blood pressure depends on their medication and lifestyle; a house price depends on its location and square footage; a credit score depends on income and payment history. These are not arbitrary correlations, they reflect underlying mechanisms. Encoding a prior over SCMs means assuming that the data-generating process has this kind of structure, which is a reasonable and well-motivated belief for most real-world tabular datasets.

An additional practical virtue: SCMs naturally produce datasets with heterogeneous features, correlated columns, and asymmetric relationships between inputs and outputs, exactly the diversity one finds in real tabular data.

2.2 Bayesian Neural Networks

A Bayesian Neural Network [2] is a neural network in which the weights are treated not as fixed parameters to be optimised, but as random variables drawn from a prior distribution. To sample a dataset from a BNN prior, one first samples a network architecture and its weights, then generates inputs x and computes outputs $y = \text{BNN}(x) + \epsilon$ for each data point.

BNNs complement SCMs in an important way. While SCMs favour datasets with interpretable causal structure, BNNs can generate smoother, more continuously-varying relationships between inputs and outputs. By mixing both priors during training, sampling datasets from SCMs with some probability and from BNNs with the rest, TabPFN’s synthetic data covers a broader range of possible data-generating mechanisms.

In practice, TabPFN V1 prior mixes SCMs and BNNs roughly equally. The V2 prior moves toward a richer, SCM-only generative pipeline with more diverse graph structures, edge types and post-processing steps, which we revisit briefly at the end.

2.3 A Prior Over These Mechanisms

Crucially, neither a single SCM nor a single BNN is fixed in advance. Instead, the prior $p(\phi)$ is a *distribution* over SCMs and BNNs, over graph structures, weight matrices, noise distributions, activation functions, numbers of nodes, and so on. Simpler structures are given higher prior probability, encoding an Occam’s razor-like preference for parsimonious explanations.

This means that when TabPFN is trained, it is exposed to an enormous variety of synthetic datasets, each generated by a different randomly sampled SCM or BNN. The space of hypotheses Φ is exactly this family of generative models, and the prior $p(\phi)$ is the distribution from which they are sampled.

3 Prior-Data Fitted Networks

We now have all the ingredients to describe Prior-Data Fitted Networks (PFNs) [3], the framework on which TabPFN is built.

3.1 The Objective

A PFN is a neural network trained to approximate the Posterior Predictive Distribution. Formally, given a prior $p(\phi)$ over data-generating mechanisms, the goal is to learn a function q_θ such that:

$$q_\theta(y_{\text{test}} \mid x_{\text{test}}, D_{\text{train}}) \approx p(y_{\text{test}} \mid x_{\text{test}}, D_{\text{train}}),$$

where the right-hand side is the true PPD defined in Section 1.3. The approximation is learned, not computed. The parameters θ are optimised so that q_θ behaves as a Bayesian predictor would, but at the cost of a single neural network forward pass rather than an intractable integral.

3.2 Synthetic Prior Fitting

How do we train q_θ ? We cannot compute the true PPD to use as a training target, but we *can sample from the prior*. This is the key insight. The training procedure, called **Synthetic Prior Fitting** (SPF), works as follows:

1. Sample a data-generating mechanism $\phi \sim p(\phi)$ (an SCM or BNN).
2. Generate a synthetic dataset $D \sim p(D | \phi)$ from that mechanism.
3. Randomly split D into a training portion D_{train} and a test portion D_{test} .
4. Train q_θ to predict the labels of D_{test} given D_{train} .

This is repeated across millions of synthetic datasets. The training loss is the expected cross-entropy:

$$\theta^* = \arg \min_{\theta} \mathbb{E}_{\phi \sim p(\phi)} \mathbb{E}_{D \sim p(D|\phi)} [-\log q_\theta(y_{\text{test}} | x_{\text{test}}, D_{\text{train}})].$$

Minimising this loss provably drives q_θ toward the true PPD [3]. Intuitively, the network has no choice but to learn to do Bayesian inference: since the training datasets are drawn from the prior and the test labels are the ground truth, the only way to minimise prediction error *in expectation* is to correctly aggregate over all mechanisms consistent with the training data.

3.3 Amortised Bayesian Inference

The SPF phase is computationally expensive, it takes weeks on multiple GPUs. But it is done *once*. Afterwards, the trained network q_θ can be applied to any new dataset without any further optimisation. The cost of Bayesian inference is paid upfront during training and then reused indefinitely. This is called **amortised Bayesian inference**: inference that is expensive to set up but essentially free to run.

This is a conceptual shift worth pausing on. Traditional machine learning fits a model to each new dataset separately. A PFN does something fundamentally different: it learns the *process of fitting* itself, across a family of datasets, and then executes that process in a single forward pass at inference time.

3.4 In-Context Learning

The mechanism by which a PFN makes predictions at inference time is a form of **in-context learning** (ICL) [4], the same phenomenon observed in large language models. The training examples D_{train} are passed as *context* directly into the network alongside the test point x_{test} , and the network produces a prediction without any weight updates. There is no gradient descent at inference time. The model adapts to the new dataset purely through attention over the provided examples.

This is different from few-shot fine-tuning: the network weights do not change. All the adaptation happens inside the forward pass, through the internal representations the model has learned to compute from the training examples.

4 TabPFN

TabPFN (Tabular Prior-data Fitted Network) [5, 6] is a PFN specifically designed for supervised prediction on tabular datasets. It instantiates the framework described above with a Transformer architecture, a carefully designed tabular prior, and an inference procedure optimised for real-world use.

4.1 Problem Setting

Consider a tabular dataset

$$D = \{(x_i, y_i)\}_{i=1}^N,$$

split into a training set $D_{\text{train}} = (X_{\text{train}}, y_{\text{train}})$ and a test set $D_{\text{test}} = (X_{\text{test}}, y_{\text{test}})$. At inference time, the model receives the labelled training set and the unlabelled test inputs, and directly estimates:

$$p(y_{\text{test}} \mid x_{\text{test}}, D_{\text{train}}).$$

No dataset-specific training occurs. No hyperparameters are tuned. The prediction is produced in a single forward pass.

4.2 Architecture

The input to TabPFN is the concatenation of the training set and the test inputs, treated as a set. Each row of the table is embedded into a vector, and the collection of row embeddings is processed by a Transformer.

The Transformer is treated here as a black box that excels at one thing: computing rich interactions between all elements of a variable-length set, regardless of order. For TabPFN, these elements are the rows of the table. The self-attention mechanism allows each row to attend to every other row, so that by the final layer, each test embedding has been informed by the entire training set. The output head then maps each test row’s embedding to a probability distribution over classes.

The V1 architecture is a 12-layer Transformer with 25.8M parameters. Importantly, test rows are only allowed to attend to training rows (not to each other), ensuring that predictions for different test points are independent, an appropriate inductive bias for supervised classification.

One non-trivial detail: TabPFN handles datasets with different numbers of features by zero-padding shorter inputs and rescaling, allowing a single trained model to generalise across arbitrary dataset shapes.

4.3 The Tabular Prior in Practice

The prior used to generate TabPFN’s synthetic training datasets is a mixture of SCMs and BNNs, as described in Section 2. A few design choices are worth highlighting:

Diversity of mechanisms. Each synthetic dataset is generated by a freshly sampled SCM or BNN. Graph structures, activation functions, noise levels, numbers of

nodes, and feature/target assignments all vary across datasets. This diversity is what allows the trained model to generalise to unseen real-world datasets.

Tabular-specific challenges. The prior explicitly includes datasets with missing values, categorical features, class imbalance, uninformative features, and outliers. Because TabPFN has seen these challenges during training (as properties of synthetic datasets), it learns strategies for handling them, not through hand-engineered pre-processing, but through the learned forward pass itself.

Simplicity bias. Simpler SCMs and BNNs (fewer nodes, smaller graphs) are sampled more often, encoding a preference for parsimonious explanations. This acts as an implicit regulariser.

4.4 Why is it Fast?

Classical machine learning algorithms require solving an optimisation problem for each new dataset:

$$\theta^* = \arg \min_{\theta} \mathcal{L}(\theta; D).$$

This involves many gradient steps and potentially extensive hyperparameter search. TabPFN bypasses this entirely. The PPD approximation is computed in one forward pass:

$$q_{\theta}(y_{\text{test}} \mid x_{\text{test}}, D_{\text{train}}) \approx p(y_{\text{test}} \mid x_{\text{test}}, D_{\text{train}}).$$

At inference time: no gradient descent, no hyperparameter optimisation, no retraining. Just a forward pass through a fixed network. On a consumer GPU, this takes on the order of seconds, even for datasets with thousands of rows. TabPFN V2 [6] reports outperforming a 4-hour tuned CatBoost ensemble in 2.8 seconds, a speedup of over $5000\times$.

4.5 Limitations

TabPFN’s design comes with inherent trade-offs. The Transformer processes all training rows simultaneously through self-attention, whose computational complexity scales quadratically with the number of samples:

$$\mathcal{O}(N^2).$$

This limits TabPFN to small and medium datasets, V1 targets up to 1 000 training samples and 100 features, and V2 pushes this to 10 000 samples and 500 features. For very large datasets, gradient-boosted methods like CatBoost [7] or XGBoost [8] remain stronger.

Additionally, the quality of TabPFN’s predictions is directly tied to how well the prior covers the real-world data at hand. If a dataset’s underlying structure is very different from anything the synthetic prior can generate, the PPD approximation will degrade. This is the fundamental limitation of any prior-based approach: the prior encodes assumptions, and assumptions can be wrong.

5 From V1 to V3: a Brief Overview

TabPFN V1 [5] established the core ideas: synthetic prior fitting, amortised Bayesian inference, and in-context learning for tabular data. Its limitations, dataset size, classification-only, sensitivity to categorical features and missing values, motivated a series of improvements.

TabPFN V2 [6], published in *Nature* in January 2025, represents the most substantial architectural revision. It introduces a **2D attention mechanism**: rather than treating each row as a single token, each cell in the table gets its own representation. Attention operates first across features within a row (feature-wise), then across rows for the same feature (sample-wise). This makes the model invariant to the ordering of both rows and columns, and allows it to generalise to larger tables than those seen during training. V2 also adds regression support, native handling of missing values and categorical features, train-state caching (enabling 300× speedups on CPU for the fit-predict regime), and a richer SCM-based prior generating around 100 million synthetic datasets per training run. The result is a model that scales to 50× larger datasets than V1 while substantially outperforming all tuned baselines.

Versions 2.5, 2.6, and 3 are best understood as scaling and engineering iterations: larger training compute budgets, expanded dataset size limits, improved ensembling strategies, and inference optimisations. The conceptual architecture is frozen at V2; what changes is how far it can be pushed in practice.

References

- [1] J. Pearl. *Causality: Models, Reasoning and Inference*, 2nd ed. Cambridge University Press, 2009.
- [2] R. M. Neal. *Bayesian Learning for Neural Networks*. Springer, 1996.
- [3] S. Müller, N. Hollmann, S. P. Arango, J. Grabocka, and F. Hutter. Transformers can do Bayesian inference. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2022.
- [4] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Nee-lakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. M. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, and D. Amodei. Language models are few-shot learners. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [5] N. Hollmann, S. Müller, K. Eggen-sperger, and F. Hutter. TabPFN: A transformer that solves small tabular classification problems in a second. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2023.
- [6] N. Hollmann, S. Müller, L. Purucker, A. Krishnakumar, M. Körfer, S. B. Hoo, R. T. Schirrmeister, and F. Hutter. Accurate predictions on small data with a tabular foundation model. *Nature*, 637:319–326, 2025.

- [7] L. Prokhorenkova, G. Gusev, A. Vorobev, A. V. Dorogush, and A. Gulin. CatBoost: unbiased boosting with categorical features. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2018.
- [8] T. Chen and C. Guestrin. XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, 2016.
- [9] N. Erickson, J. Mueller, A. Shirkov, H. Zhang, P. Larroy, M. Li, and A. Smola. AutoGluon-Tabular: Robust and accurate AutoML for structured data. arXiv preprint arXiv:2003.06505, 2020.